Action Space: 可选择的动作，比如 {left，up，right}

离散的动作

Policy: 策略函数，输入State，输出Action的概率分布。一般用π表示。

$\pi(left|s_t) = 0.1$
$\pi(up|s_t) = 0.2$
$\pi(right|s_t) = 0.7$

神经网络要拟合的东西

Trajectory: 轨迹,用τ表示， 一连串状态和动作的序列。Episode,Rollout。 $\{s_0, a_0, s_1, a_1, \ldots\}$

$s_{t+1} = f(s_t, a_t)$ 确定
$s_{t+1} = P(\cdot|s_t, a_t)$ 随机

一局游戏从开始到结束的动作序列

Return: 回报，从当前时间点到游戏结束的Reward的累积和。

强化学习的目标：

训练一个神经网络π，使得神经网络在所有的可能的trajectory中得到的期望值最大

Return

⇓

$E(R(\tau))_{\tau\sim P_\theta(\tau)} = \sum_\tau R(\tau)P_\theta(\tau)$

这就是期望的表达式

由于要优化梯度，故对θ求导

$$\begin{aligned}
\nabla E(R(\tau))_{\tau\sim P_\theta(\tau)} &= \nabla \sum_\tau R(\tau)P_\theta(\tau) \\
&= \sum_\tau R(\tau)\nabla P_\theta(\tau) \\
&= \sum_\tau R(\tau)\nabla P_\theta(\tau)\frac{P_\theta(\tau)}{P_\theta(\tau)} \\
&= \sum_\tau P_\theta(\tau)R(\tau)\frac{\nabla P_\theta(\tau)}{P_\theta(\tau)} \quad \text{构造成 } E=\Sigma p(x)\cdot x \text{ 的形式} \\
&= \sum_\tau P_\theta(\tau)R(\tau)\frac{\nabla P_\theta(\tau)}{P_\theta(\tau)} \quad \text{用实际的数据进行估计} \\
&\approx \frac{1}{N}\sum_{n=1}^{N} R(\tau^n)\frac{\nabla P_\theta(\tau^n)}{P_\theta(\tau^n)} \qquad \nabla \log f(x) = \frac{\nabla f(x)}{f(x)} \\
&= \frac{1}{N}\sum_{n=1}^{N} R(\tau^n)\nabla \log P_\theta(\tau^n) \qquad \tau\sim P_\theta(\tau) \\
&\quad \text{用对数函数进行化简} \\
&= \frac{1}{N}\sum_{n=1}^{N} R(\tau^n)\nabla \log P_\theta(\tau^n) \\
&= \frac{1}{N}\sum_{n=1}^{N} R(\tau^n)\nabla \log \prod_{t=1}^{T_n} P_\theta(a_n^t|s_n^t) \quad \text{→ 轨迹τ是各个状态叠加而来，故用连乘} \\
&= \frac{1}{N}\sum_{n=1}^{N} R(\tau^n)\sum_{t=1}^{T_n} \nabla \log P_\theta(a_n^t|s_n^t) \quad \text{→ } \log\Pi = \Sigma\log \\
&= \frac{1}{N}\sum_{n=1}^{N}\sum_{t=1}^{T_n} R(\tau^n)\nabla \log P_\theta(a_n^t|s_n^t)
\end{aligned}$$

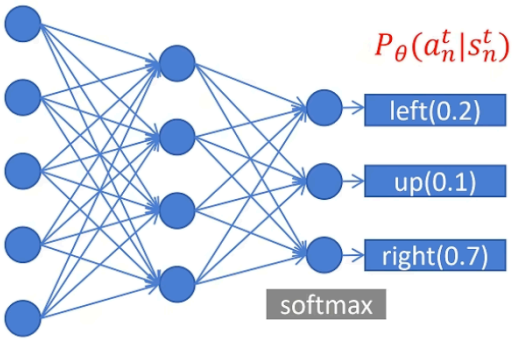
$\log = \frac{1}{N}\sum_{n=1}^{N}\sum_{t=1}^{T_n} R(\tau^n)\log P_\theta(a_n^t|s_n^t)$

→ R>0 后面要尽量大
R<0 后面要尽量小

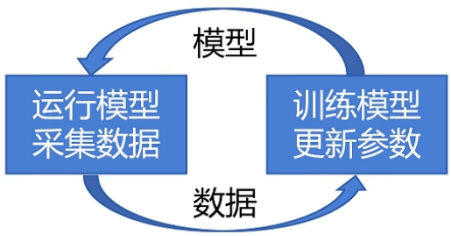
→ 利用全局的R对历史的每一步决策都进行优化

习惯最小化
故加负号

$$Loss=-\frac{1}{N}\sum_{n=1}^{N}\sum_{t=1}^{T_n}R(\tau^n)\log P_\theta(a_n^t|s_n^t)$$



$$R(\tau^n)$$
$$\tau^1 -> R(\tau^1)$$
$$\tau^2 -> R(\tau^2)$$
$$\vdots$$
$$\tau^n -> R(\tau^n)$$



On Policy

这种方式太慢了.而且有漏洞.
纯结果导向错误地打压了中相对好的Action

PPO 改进.

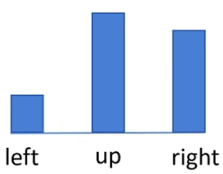
$$=\frac{1}{N}\sum_{n=1}^{N}\sum_{t=1}^{T_n}R(\tau^n)\nabla\log P_\theta(a_n^t|s_n^t)$$

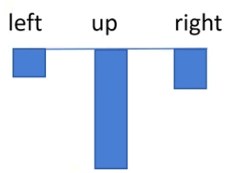
$$R(\tau^n)\rightarrow\sum_{t'=t}^{T_n}\gamma^{t'-t}r_{t'}^n=R_t^n$$

做出某个 action 后,应该只对 action 之后的 reward 有影响.即

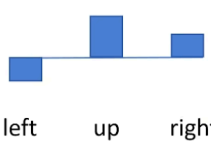
$$=\frac{1}{N}\sum_{n=1}^{N}\sum_{t=1}^{T_n}R_t^n\nabla\log P_\theta(a_n^t|s_n^t)$$

$$=\frac{1}{N}\sum_{n=1}^{N}\sum_{t=1}^{T_n}(R_t^n-B(s_n^t))\nabla\log P_\theta(a_n^t|s_n^t)$$

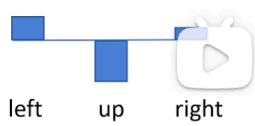




好的局势



坏的局势



为了让模型知道这些不同的策略之间相对的好坏，

引出以下3个function

Action-Value Function

R_t^n 每次都是一次随机采样，方差很大，训练不稳定。

$Q_\theta(s,a)$在state s下，做出Action a，期望的回报。动作价值函数。] 这个状态下做出行为a所带来的期望收益

State-Value Function

$V_\theta(s)$在state s下，期望的回报。状态价值函数。] 这种状态下所带来的期望的回报（均值）

Advantage Function

$A_\theta(s,a) = Q_\theta(s,a) - V_\theta(s)$ 在state s下，做出Action a，比其他动作能带来多少优势。

作差，这样就计算出了action相对的好坏

$$\frac{1}{N}\sum_{n=1}^{N}\sum_{t=1}^{T_n} A_\theta(s_n^t, a_n^t)\nabla \log P_\theta(a_n^t|s_n^t)$$

$A_\theta(s,a) = Q_\theta(s,a) - V_\theta(s)$

$Q_\theta(s,a)$在state s下，做出Action a，期望的回报。动作价值函数。

$Q_\theta(s_t,a) = r_t + \gamma * V_\theta(s_{t+1})$

$A_\theta(s_t,a) = r_t + \gamma * V_\theta(s_{t+1}) - V_\theta(s_t)$

$V_\theta(s_{t+1}) \approx r_{t+1} + \gamma * V_\theta(s_{t+2})$

$$A_\theta^1(s_t,a) = r_t + \gamma * V_\theta(s_{t+1}) - V_\theta(s_t)$$

$$A_\theta^2(s_t,a) = r_t + \gamma * r_{t+1} + \gamma^2 * V_\theta(s_{t+2}) - V_\theta(s_t)$$

$$A_\theta^3(s_t,a) = r_t + \gamma * r_{t+1} + \gamma^2 * r_{t+2} + \gamma^3 V_\theta(s_{t+3}) - V_\theta(s_t)$$

$$\vdots$$

$$A_\theta^T(s_t,a) = r_t + \gamma * r_{t+1} + \gamma^2 * r_{t+2} + \gamma^3 * r_{t+3} + \cdots + \gamma^T * r_T - V_\theta(s_t)$$

偏差 ↓ 方差 ↑

$$\delta_t^V = r_t + \gamma * V_\theta(s_{t+1}) - V_\theta(s_t)$$

$$\delta_{t+1}^V = r_{t+1} + \gamma * V_\theta(s_{t+2}) - V_\theta(s_{t+1})$$

$$A_\theta^1(s_t, a) = \delta_t^V$$

$$A_\theta^2(s_t, a) = \delta_t^V + \gamma\delta_{t+1}^V$$

$$A_\theta^3(s_t, a) = \delta_t^V + \gamma\delta_{t+1}^V + \gamma^2\delta_{t+2}^V$$

Generalized Advantage Estimation (GAE)

$$A_\theta^{GAE}(s_t, a) = (1-\lambda)(A_\theta^1 + \lambda * A_\theta^2 + \lambda^2 A_\theta^3 + \cdots) \qquad \lambda = 0.9: \quad A_\theta^{GAE} = 0.1A_\theta^1 + 0.09A_\theta^2 + 0.081A_\theta^3 + \cdots$$

$$= (1-\lambda)(\delta_t^V + \lambda * (\delta_t^V + \gamma\delta_{t+1}^V) + \lambda^2(\delta_t^V + \gamma\delta_{t+1}^V + \gamma^2\delta_{t+2}^V) + \cdots)$$

$$= (1-\lambda)(\delta_t^V(1 + \lambda + \lambda^2 + \cdots) + \gamma\delta_{t+1}^V * (\lambda + \lambda^2 + \cdots) + \cdots)$$

$$= (1-\lambda)(\delta_t^V \frac{1}{1-\lambda} + \gamma\delta_{t+1}^V \frac{\lambda}{1-\lambda} + \cdots)$$

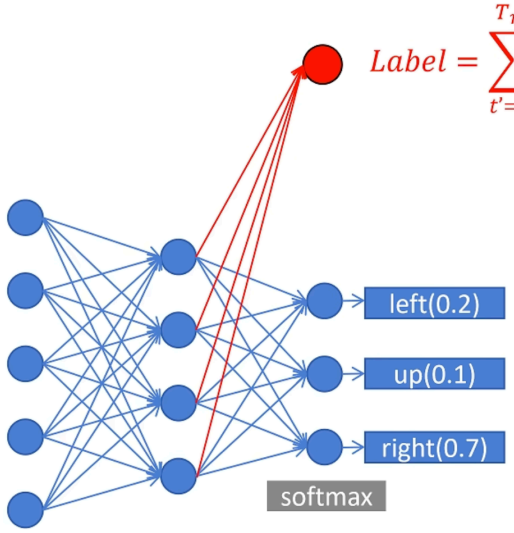
$$= \sum_{b=0}^{\infty} (\gamma\lambda)^b \, \delta_{t+b}^V$$

$$\delta_t^V = r_t + \gamma * V_\theta(s_{t+1}) - V_\theta(s_t)$$

$$A_\theta^{GAE}(s_t, a) = \sum_{b=0}^{\infty} (\gamma\lambda)^b \, \delta_{t+b}^V$$

$$\frac{1}{N}\sum_{n=1}^{N}\sum_{t=1}^{T_n} A_\theta^{GAE}(s_n^t, a_n^t)\nabla \log P_\theta(a_n^t|s_n^t)$$

$$Label = \sum_{t'=t}^{T_n} \gamma^{t'-t} r_{t'}^n$$



$$\frac{1}{N}\sum_{n=1}^{N}\sum_{t=1}^{T_n} A_{\theta}^{GAE}(s_n^t, a_n^t)\nabla \log P_{\theta}(a_n^t|s_n^t)$$

$$\nabla \log f(x) = \frac{\nabla f(x)}{f(x)}$$

其他同学的行为 你与其他同学做这样的差异

$$= \frac{1}{N}\sum_{n=1}^{N}\sum_{t=1}^{T_n} A_{\theta'}^{GAE}(s_n^t, a_n^t)\frac{P_{\theta}(a_n^t|s_n^t)}{P_{\theta'}(a_n^t|s_n^t)}\nabla \log P_{\theta}(a_n^t|s_n^t)$$

使用一个 reference policy

$$= \frac{1}{N}\sum_{n=1}^{N}\sum_{t=1}^{T_n} A_{\theta'}^{GAE}(s_n^t, a_n^t)\frac{\cancel{P_{\theta}(a_n^t|s_n^t)}}{P_{\theta'}(a_n^t|s_n^t)}\frac{\nabla P_{\theta}(a_n^t|s_n^t)}{\cancel{P_{\theta}(a_n^t|s_n^t)}}$$

并行协同训练

$$= \frac{1}{N}\sum_{n=1}^{N}\sum_{t=1}^{T_n} A_{\theta'}^{GAE}(s_n^t, a_n^t)\frac{\nabla P_{\theta}(a_n^t|s_n^t)}{P_{\theta'}(a_n^t|s_n^t)}$$

$$\text{Loss} = -\frac{1}{N}\sum_{n=1}^{N}\sum_{t=1}^{T_n} A_{\theta'}^{GAE}(s_n^t, a_n^t)\frac{P_{\theta}(a_n^t|s_n^t)}{P_{\theta'}(a_n^t|s_n^t)}$$ PPO Loss 核心

$$Loss_{ppo} = -\frac{1}{N}\sum_{n=1}^{N}\sum_{t=1}^{T_n} A_{\theta'}^{GAE}(s_n^t, a_n^t)\frac{P_{\theta}(a_n^t|s_n^t)}{P_{\theta'}(a_n^t|s_n^t)} + \beta KL(P_{\theta}, P_{\theta'})$$ 防止相差太大

$$Loss_{ppo2} = -\frac{1}{N}\sum_{n=1}^{N}\sum_{t=1}^{T_n} min(A_{\theta'}^{GAE}(s_n^t, a_n^t)\frac{P_{\theta}(a_n^t|s_n^t)}{P_{\theta'}(a_n^t|s_n^t)}, clip(\frac{P_{\theta}(a_n^t|s_n^t)}{P_{\theta'}(a_n^t|s_n^t)}, 1-\varepsilon, 1+\varepsilon)A_{\theta'}^{GAE}(s_n^t, a_n^t))$$

另外一种 PPO 算法的实现